I have a set of algorithms that connect points of interest in a 2D map to one another with paths. The algorithms utilize things like a* and k-nearest neighbors to make sure that all points of interest get connected somehow. However, I want to minimize the amount of actual "path" used to do this. Meaning, I'd like to use some kind of optimization algorithm (or maybe something else) to check for areas where paths can be combined, and thereby reduce inefficiencies like having two paths run directly parallel.

What you're describing is a classic graph optimization problem. Right now, your pipeline ("connect everything somehow using A* + k-NN") guarantees connectivity, but not efficiency. The moment you say "minimize total path length and merge redundant routes," you've essentially stepped into the territory of **network optimization** problems.

Here are the most relevant approaches, from simplest to more advanced:

1. Minimum Spanning Tree (MST) — your baseline fix

If your only goal is:

> "Connect all points with the least total path length"

Then you want a **Minimum Spanning Tree**.

- Algorithms: Kruskal's Algorithm or Prim's Algorithm
- Input: a graph where edge weights = path cost (from A*, Euclidean distance, etc.)
- Output: a tree that connects all nodes with **no redundant edges**

Why this helps

- Eliminates parallel/redundant paths automatically
- Guarantees minimal total edge weight
- Very fast and simple

Limitation

- Produces a **tree**, so:
  - No loops
  - Might not follow your existing terrain-aware A* paths unless you precompute them as edges

👉 Practical trick:

- Use A* to compute shortest paths between nearby nodes
- Build a graph from those distances
- Run MST on top

2. Steiner Tree — what you *actually* want

Your problem sounds even closer to a **Steiner Tree Problem**.

This extends MST by allowing:

- **New intermediate points (Steiner points)**
- These let paths merge naturally (like a "Y" junction instead of two parallel lines)

Why it's powerful

- Produces networks that look like:
  - Road systems
  - River tributaries
  - Efficient branching structures
- Minimizes total length **better than MST**

Tradeoff

- NP-hard (exact solution is expensive)
- Usually solved with heuristics or approximations

👉 In practice:

- Use approximation algorithms (e.g. 2-approx Steiner tree)
- Or iterative merging heuristics (see below)

3. Post-processing / path merging (very practical for your setup)

Since you already have paths, you can **optimize after the fact**:

A. Detect parallel / overlapping segments

- Represent paths as polylines
- For each pair:
    - Check if segments are:
        - Close (distance threshold)
        - Roughly parallel (angle threshold)

B. Merge them

- Replace both with:
    - A shared “spine” path
    - Short connectors to endpoints

This is essentially a **geometric clustering problem**:

- Cluster nearby segments
- Replace clusters with a single representative

4. Graph simplification via edge contraction

Convert your paths into a graph:

- Nodes = intersections/endpoints
- Edges = path segments

Then:

- Merge nodes within a small radius
- Collapse degree-2 nodes (straight lines)
- Combine parallel edges

This is similar to mesh/road-network simplification.

5. Flow-based optimization (more advanced)

You can model this as:

- Nodes = POIs
- Possible edges = candidate paths
- Cost = path length

Then solve using:

- Minimum-cost flow
- Integer linear programming (ILP)

This gives you:

- Fine control (penalize redundancy, enforce structure)

But it's heavier than you likely need.

6. A hybrid approach (what I'd recommend)

Given your current system:

Step 1 — Generate candidates

- Use A* + k-NN (what you already do)

Step 2 — Build graph

- Nodes = POIs
- Edge weights = path cost

Step 3 — Run MST

- Removes obvious redundancy

Step 4 — Improve with Steiner-like merging

- Detect nearby edges
- Introduce junction points
- Re-route locally

Key intuition shift

Right now you're thinking:

> "Connect points with paths"

But optimization requires:

> "Design a *network* that shares infrastructure wherever possible"

That shift—from independent paths → shared graph—is the core idea.

If you want to go deeper

I can:

- Sketch pseudocode for a Steiner-like approximation
- Help you implement path-merging geometry
- Or suggest spatial indexing tricks (KD-trees, R-trees) for fast detection

Just tell me 👍

My assignment requires that I do *some* kind of optimization, so maybe overkill is actually the move to make here. What would a flow-based optimization look like here?

For clearer context, currently I'm working with something a lot like Steps 1 & 2 that you described in part 6: I generate candidates and then build a graph with nodes and I have

data structures in place for defining path costs.

If “overkill” is actually a requirement, then a flow-based or ILP-style formulation is a great choice—it’s rigorous, flexible, and very defensible academically.

Let’s walk through what this would look like in your case.

1. Core idea: turn your map into a decision problem

You already have:

- Nodes = POIs
- Candidate edges = paths (from A*)
- Cost = path length (or weighted cost)

Now instead of *just building a graph*, you ask:

> “Which subset of these edges should I keep so that everything is connected, while minimizing total cost?”

That becomes an optimization problem.

2. Decision variables

Define a binary variable for each edge:

- $x_e = 1$ → edge is used
- $x_e = 0$ → edge is not used

3. Objective function

Minimize total path cost:

$$\text{minimize} \sum_{e \in E} c_e \cdot x_e$$

This alone already pushes the system to:

- Remove redundant paths

- Prefer shared routes over parallel ones

4. Connectivity constraint (this is the interesting part)

You need to enforce:

> "All POIs must be connected somehow"

There are a few ways to do this, but the most "flow-based" is:

Single-commodity flow formulation

Pick one node as a **root**.

Then:

- Send 1 unit of flow from the root to every other node
- Flow can only travel on edges you "activate"

Variables:

- f_{uv} = flow on edge (u → v)

Constraints:

A. Flow conservation

For each node:

- Root: sends out $n - 1$
- Others: receive 1

B. Capacity constraint

Flow can only exist if the edge is selected:

$$f_{uv} \leq (n-1) \cdot x_e$$

What this does

- Forces the chosen edges to form a **connected structure**
- Automatically eliminates disconnected subgraphs

- Prevents useless parallel edges unless they actually help routing

5. Why this reduces parallel paths

Suppose you have two nearly identical parallel paths:

- Both connect the same regions
- One is slightly cheaper

The optimizer will:

- Route all flow through the cheaper one
- Set the other $x_e = 0$

👉 That's exactly the behavior you want.

6. Adding "anti-parallel" penalties (this is where you shine)

You can go beyond MST behavior by explicitly discouraging redundancy.

Idea:

If two edges are:

- Close together
- Similar direction

Add a penalty if both are selected.

Example:
Introduce:

$$x_e + x_{e'} \leq 1 \quad (\text{hard constraint})$$

or softer:

$$\text{objective} += \lambda \cdot x_e \cdot x_{e'}$$

This pushes the solver to:

- Merge paths
- Share infrastructure

7. Even more overkill: multi-commodity flow

Instead of one root:

- Each node sends flow to every other node

This:

- Models “all-pairs connectivity”
- Strongly encourages shared routes

But:

- Much more expensive computationally
- Probably unnecessary unless you want to impress

8. Practical implementation

You’d typically solve this using:

- Google OR-Tools
- Gurobi
- PuLP

Workflow:

1. Generate candidate edges (your current system)
2. Build ILP:
    - Variables: x_e, f_{uv}
    - Constraints: flow + capacity
3. Solve
4. Extract selected edges

9. What you’ll get (and how it looks)

Compared to your current system:

Before

- Many redundant paths
- Parallel edges
- Over-connected graph

After

- Tree-like or near-tree structure
- Shared “highways”
- Clean branching network

10. If you want to push it further (extra credit territory)

You can add:

- **Edge capacity costs** → simulate “roads getting wider instead of duplicated”
- **Node penalties** → discourage too many intersections
- **Geometric penalties** → smoother paths

At that point, you’re basically solving a simplified version of real-world road network design.

TL;DR

A flow-based optimization for your problem:

- Binary edge selection variables
- Minimize total cost
- Enforce connectivity via flow constraints
- Optionally penalize parallel edges

It’s essentially a generalized, more flexible version of an MST—with knobs you can tune.

If you want, I can:

- Write a concrete Python + OR-Tools example
- Show how to detect “parallel edges” geometrically
- Or help you scale it so it doesn’t explode computationally

Okay chat, I'm liking what I'm hearing. Here's the

gameplan: I'm going to show you my code, and you're going to help me implement flow-based, multi-commodity flow, where we try to send flow from every node to every other, build up a cost function, and try to minimize:
1. distance traveled
2. total paths placed in the map
3. sharp corners taken at each node/intersection

It's totally okay for this to be computationally expensive. Does that sound good?

That sounds like a *fantastic* plan—and honestly, exactly the kind of "ambitious but justifiable" overkill that makes a project stand out.

You're essentially building a **network design optimizer with geometric awareness**, which is way beyond a basic MST.

Let me sanity-check and slightly refine your plan so it works cleanly when we translate it into code.

✅ Your formulation (refined)

You want a **multi-commodity flow model** where:

- Every node sends flow to every other node
- Flow can only travel on selected edges
- The solver decides which edges to "activate"

And you minimize:

1. Distance traveled

$$\sum c_e \cdot x_e$$

✔ Standard, no issues.

2. Total number of paths (edge count penalty)

$$\lambda_1 \sum x_e$$

✔️ This is great because:

- It discourages unnecessary edges
- Pushes toward shared infrastructure

3. Sharp corners (this is the fun part)

This is where things get interesting.

You'll need to model **turn penalties at nodes**.

Concept:

At a node v, if flow goes:

- from edge (u → v)
- to edge (v → w)

Then you penalize the **angle between those edges**.

⚠️ Important modeling detail

Flow alone doesn't track *turns*. You need to explicitly represent them.

Solution: edge-pair variables

Define variables like:

- $y_{u,v,w} = 1$ if flow goes through node v from $u \to v \to w$

Then:

$$\text{corner cost} = \sum \theta_{u,v,w} \cdot y_{u,v,w}$$

Where:

- $\theta_{u,v,w}$ = angle penalty (precomputed)

🔥 This is the key insight

You are no longer optimizing just edges—you are optimizing **paths through nodes**.

That's what makes this model powerful (and expensive).

🧠 Multi-commodity flow structure

For each pair of nodes (s, t):

- Define flow f_{uv}^k where $k = (s, t)$

Constraints:

1. Flow conservation

- Source sends 1
- Sink receives 1
- Others pass flow through

2. Edge activation constraint

$$f_{uv}^k \leq x_{uv}$$

⚖️ Final objective

Your full objective becomes:

$$\min \left(\sum c_e x_e + \lambda_1 \sum x_e + \lambda_2 \sum \theta_{u,v,w} y_{u,v,w} \right)$$

🚨 Reality check (important before coding)

This model will explode in size:

- Commodities: $O(n^2)$
- Flow vars: $O(n^2 \cdot |E|)$
- Turn vars: potentially huge

👉 Totally fine since you said that's acceptable—but we'll want to:

- Start small (like 10–20 nodes)
- Possibly prune candidate edges (k-NN helps a lot)

🛠️ Implementation strategy (what we'll do together)

When you send your code, I'll help you:

Step 1 — Integrate optimization layer

- Keep your current graph generation
- Build ILP on top

Step 2 — Add variables

- Edge selection x_e
- Flow variables f_{uv}^k
- Turn variables $y_{u,v,w}$

Step 3 — Precompute geometry

- Edge directions
- Angles at nodes

Step 4 — Build constraints cleanly

- Flow conservation
- Edge activation
- Turn consistency

Step 5 — Tune weights

- Distance vs edge count vs smoothness

💡 One suggestion (optional but smart)

You *might* start with:

- Single-commodity flow (one root)
- - turn penalties

Then upgrade to multi-commodity once everything works.

But if you want maximum impact, going straight to multi-commodity is totally defensible.

👍 Bottom line

Yes—this is:

- Technically deep ✅
- Visually meaningful ✅
- Easy to justify in a report ✅
- Impressive if it works ✅

Send over your code whenever you're ready, and we'll start wiring this into a real solver step-by-step.

location_generator.py (This is a file that generates our 2D maps, which contain buildings, doors, and points of interest. Interiors of buildings can be traversed, but should be at a cost. This code doesn't generate any paths, that's what the next file will do):

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

##############################################################
### NOTE: COORDS ARE IN (LAT, LON) MEANING (Y, X) ###
### THAT IS HOW MATRICES HANDLE IT, AND ALSO MAPS ###
##############################################################

# options for materials. If you add one, add it to the colormap too
material = {
    "open": 0,
```

```
    "paved": 1,
    "door": 2,
    "blocked": 3,
    "interior": 4,
    "poi": 5
}

color_map = {
    0: (0.9, 0.95, 0.9),    # open
    1: (0.7, 0.7, 0.8),     # paved
    2: (0.9, 0.5, 0.3),     # door
    3: (0.1, 0.2, 0.3),     # blocked
    4: (0.6, 0.6, 0.7),     # interior
    5: (0.9, 0.6, 0.7)      # point of interest (staircase, quad)
}

# https://www.latlong.net/
# try to keep the bottom left corner first and top right
second. It doesn't matter for the code,
# but it can help with making sure the doors are properly
aligned with the buildings
buildings = {
    "eb": ((40.246081,-111.648444),
(40.246474,-111.647306)),
    "eb-tunnel": ((40.246355,-111.648266),
(40.246862,-111.648201)),
    "clyde1": ((40.246591,-111.648354),
(40.247315,-111.647668)),
    "clyde2": ((40.246698,-111.648447),
(40.247111,-111.647831)),
    "marb": ((40.246622,-111.649448),
(40.247032,-111.648960)),
    "kennedy1": ((40.247321,-111.649504),
(40.247747,-111.649274)),
    "kennedy2": ((40.247673,-111.649504),
(40.247850,-111.648971)),
    "bike racks": ((40.247416,-111.648414),
(40.247878,-111.648414)),
    "wilk1": ((40.248023,-111.648414),
(40.248707,-111.647110)),
```

```
    "wilk2": ((40.248236,-111.647775),
(40.249063,-111.646353)),
    "wilk parking": ((40.246840,-111.647523),
(40.247830,-111.64656)),
    "hbll1": ((40.248073,-111.649679),
(40.248422,-111.648825)),
    "hbll2": ((40.248229,-111.649385),
(40.249363,-111.649116)),
    "hbll3": ((40.248593,-111.649754),
(40.248999,-111.648739)),
}

# bridges from interiors to exteriors. Make CERTAIN these
fall on the exact lines of the exteriors they touch
doors = {
    "eb": [(40.246253,-111.648444)],
    "clyde": [(40.247062,-111.648447),
(40.246739,-111.648447), (40.247315,-111.648222)],
    "marb": [(40.246622,-111.649200),
(40.246830,-111.648960), (40.247032,-111.649200),
            (40.246830,-111.649448)],
    "kennedy": [(40.247321,-111.649395),
(40.247573,-111.649504), (40.247850,-111.649400),
                (40.247850,-111.649094),
(40.247580,-111.649274)],
    "wilk": [(40.248062,-111.648414),
(40.248023,-111.647389), (40.248833,-111.647775),
            (40.248707,-111.648289)],
    "wilk parking sidewalk points":
[(40.247830,-111.647389), (40.247391,-111.647523)],
    "library": [(40.248073,-111.649249),
(40.249227,-111.649385), (40.249227,-111.649116)]
}

# points of interest that aren't doors
poi = {
    "south street": [(40.245824,-111.649303),
(40.245877,-111.649180),
                    (40.245992,-111.648810),
(40.246008,-111.648579), (40.245957,-111.647407)],
```

```
    "central strip": [(40.247971,-111.648609),
(40.247971,-111.648838),
                      (40.247971,-111.649251),
(40.247971,-111.649999)],
    "little study zone": [(40.247759,-111.648712)],
}


def simple_space(y, x, mode):
    """Generates a rectangular space with doors in the
corners."""
    array = np.zeros((y, x), dtype=np.uint8)
    # doors in the corners
    if (mode == "corners"):
        array[0, 0] = material["door"]
        array[y-1, 0] = material["door"]
        array[0, x-1] = material["door"]
        array[y-1, x-1] = material["door"]
    elif (mode == "corridor"):
        array[y//2, 0] = material["door"]
        array[y//2, x-1] = material["door"]

    return array


def building(map, p1, p2):
    """Alters the incoming map by adding a blocked zone
between the specified points."""
    y1, x1 = p1
    y2, x2 = p2

    miny = np.minimum(y1, y2) - 0.5
    maxy = np.maximum(y1, y2) + 0.5
    minx = np.minimum(x1, x2) - 0.5
    maxx = np.maximum(x1, x2) + 0.5

    w = maxy - miny
    h = maxx - minx
    # a building, bottom-left is (miny, minx)
    r, c = np.indices(map.shape)
```

```
    # only put down a building where there is not already
one
    mask1 = (np.abs(r - (w/2 + miny)) < w/2) & (np.abs(c -
(h/2 + minx)) < h/2) & (map == material["open"])
    map[mask1] = material["blocked"]

    # fill the interiors of buildings with walkable spaces
    mask2 = (np.abs(r - (w/2 + miny)) < w/2 - 1) & (np.abs(c
- (h/2 + minx)) < h/2 - 1)
    map[mask2] = material["interior"]

    return map


def cost(map, units):
    """Returns the total number of paved spaces in the map,
multiplied by the cost per paved square"""
    return np.count_nonzero(map ==
material["paved"])*units


def print_map(map):
    """Prints the map out in plain text, for the small ones"""
    print(np.flip(map, 0))


def plot_map(map):
    """Plots the map nice and pretty with colors and such"""
    cmap = ListedColormap([color_map[i] for i in
sorted(color_map.keys())])

    plt.figure()
    plt.imshow(map, cmap=cmap, origin="lower", vmin=0,
vmax=len(color_map)-1)
    plt.xticks([])
    plt.yticks([])
    plt.gca().set_aspect('equal')
    plt.show()
```

```
def campus(resolution, p1, p2):
    """Builds a map of campus between two coordinates"""
    y1, x1 = p1
    y2, x2 = p2

    miny = np.minimum(y1, y2)
    maxy = np.maximum(y1, y2)
    minx = np.minimum(x1, x2)
    maxx = np.maximum(x1, x2)

    w = maxy - miny
    h = maxx - minx

    # convert from lat/long to scaled, integer coordinates
    mind = np.minimum(w, h)
    scale = int(resolution/mind)
    w = int(w*scale)
    h = int(h*scale)

    map = np.zeros((w, h), dtype=np.uint8)

    for c1, c2 in buildings.values():
        point1 = (int(scale*(c1[0] - miny)), int(scale*(c1[1] -
minx)))
        point2 = (int(scale*(c2[0] - miny)), int(scale*(c2[1] -
minx)))
        map = building(map, point1, point2)

    for door_list in doors.values():
        for d in door_list:
            y = int(scale*(d[0] - miny))
            x = int(scale*(d[1] - minx))

            # don't plot a door if it's index is negative
            if 0 <= y < map.shape[0] and 0 <= x <
map.shape[1]:
                map[y, x] = material["door"]

    for poi_list in poi.values():
        for p in poi_list:
```

```
            y = int(scale*(p[0] - miny))
            x = int(scale*(p[1] - minx))

            # don't plot a door if it's index is negative
            if 0 <= y < map.shape[0] and 0 <= x <
map.shape[1]:
                map[y, x] = material["poi"]

    return map


def visitables(map):
    """Returns a list of points that must be visitable (doors
and pois)"""
    spots = np.argwhere(np.isin(map, [material["door"],
material["poi"]]))
    return spots

if __name__ == "__main__":
    # basic = simple_space(6, 6, "corners")
    # print_array(basic)
    # print(cost(basic, 10))
    # plot_grid(basic)

    # basic = simple_space(5, 5, "corridor")
    # plot_grid(basic)

    campus_plot = campus(64, (40.245751,-111.649794),
(40.248344,-111.646590))
    plot_map(campus_plot)
    # print(visitables(campus_plot).shape)
```

File number two: path_generator.py (This has a bunch of different options for generating paths between buildings and points of interest. Right now it has a straight-line generator that can be used for maximal straight-line connectivity, as well as some k-nearest neighbors and some a*):

```
from location_generator import *
```

```
import math as m
import heapq

def bresenham_line(x0, y0, x1, y1):
    """return list of grid cells between two points
(inclusive)."""
    cells = []

    dx = abs(x1 - x0)
    dy = abs(y1 - y0)

    x, y = x0, y0
    sx = 1 if x1 > x0 else -1
    sy = 1 if y1 > y0 else -1

    if dx > dy:
        err = dx / 2.0
        while x != x1:
            cells.append((x, y))
            err -= dy
            if err < 0:
                y += sy
                err += dx
            x += sx
    else:
        err = dy / 2.0
        while y != y1:
            cells.append((x, y))
            err -= dx
            if err < 0:
                x += sx
                err += dy
            y += sy

    cells.append((x1, y1))
    return cells


def connect_local_pois(grid, max_distance):
    visited_pairs = set()
```

```
    pois = visitables(grid)

    for i, poi_a in enumerate(pois):
        for j, poi_b in enumerate(pois):
            if i >= j:
                continue    # avoid duplicates + self
            elif (i, j) in visited_pairs:
                continue    # avoid repeats
            elif m.dist(poi_a, poi_b) > max_distance:
                continue    # avoid out of bounds

            line_cells = bresenham_line(poi_a[0], poi_a[1],
poi_b[0], poi_b[1])

            # handle blockages
            blocked = False
            for (x, y) in line_cells:
                if grid[x][y] == material["blocked"]:
                    blocked = True
                    break

            if blocked:
                continue  # avoid blocked connections

            # draw the path
            for (x, y) in line_cells:
                if grid[x][y] == material["open"]:
                    grid[x][y] = material["paved"]
            visited_pairs.add((i, j))

    return grid

def connect_nearest_pois(grid, k):
    visited_pairs = set()
    pois = visitables(grid)

    for i, poi_a in enumerate(pois):
        # compute distances to all other POIs
        distances = []
        for j, poi_b in enumerate(pois):
```

```
            if i == j:
                continue
            dist = m.dist(poi_a, poi_b)
            distances.append((dist, j, poi_b))

        # sort by distance
        distances.sort(key=lambda x: x[0])

        connections_made = 0

        for _, j, poi_b in distances:
            if connections_made >= k:
                break

            if (i, j) in visited_pairs:
                continue  # avoid duplicates

            line_cells = bresenham_line(poi_a[0], poi_a[1],
poi_b[0], poi_b[1])

            # check for blockages
            blocked = False
            for (x, y) in line_cells:
                if grid[x][y] == material["blocked"]:
                    blocked = True
                    break

            if blocked:
                continue  # skip and try next nearest

            # draw the path
            for (x, y) in line_cells:
                if grid[x][y] == material["open"]:
                    grid[x][y] = material["paved"]

            visited_pairs.add((i, j))
            connections_made += 1

    return grid
```

```
def astar(grid, start, goal):
    rows, cols = len(grid), len(grid[0])

    def heuristic(a, b):
        # Manhattan distance (good for grids)
        return abs(a[0] - b[0]) + abs(a[1] - b[1])

    def neighbors(x, y):
        for dx, dy in [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(1,1),(-1,1),
(1,-1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < rows and 0 <= ny < cols:
                yield nx, ny, dx, dy

    def movement_cost(nx, ny, dx, dy):
        cell = grid[nx][ny]

        if dx != 0 and dy != 0:
            cost = m.sqrt(2)
        else:
            cost = 1.0

        if cell == material["paved"] or cell ==
material["interior"]:
            cost *= 0

        return cost

    open_set = []
    heapq.heappush(open_set, (0, start))

    came_from = {}
    g_score = {start: 0}

    while open_set:
        _, current = heapq.heappop(open_set)

        if current == goal:
            # reconstruct path
```

```
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path

        for nx, ny, dx, dy in neighbors(*current):
            if grid[nx][ny] == material["blocked"]:
                continue

            tentative_g = g_score[current] +
movement_cost(nx, ny, dx, dy)

            if (nx, ny) not in g_score or tentative_g <
g_score[(nx, ny)]:
                came_from[(nx, ny)] = current
                g_score[(nx, ny)] = tentative_g
                f_score = tentative_g + heuristic((nx, ny), goal)
                heapq.heappush(open_set, (f_score, (nx, ny)))

    return None  # no path found


def connect_nearest_pois_astar(grid, k):
    visited_pairs = set()
    pois = visitables(grid)

    for i, poi_a in enumerate(pois):
        distances = []
        for j, poi_b in enumerate(pois):
            if i == j:
                continue
            dist = m.dist(poi_a, poi_b)
            distances.append((dist, j, poi_b))

        distances.sort(key=lambda x: x[0])

        connections_made = 0
```

```
            for _, j, poi_b in distances:
                if connections_made >= k:
                    break

                if (i, j) in visited_pairs or (j, i) in visited_pairs:
                    continue

                path = astar(grid, tuple(poi_a), tuple(poi_b))

                if path is None:
                    continue  # no valid route

                # draw the path
                for (x, y) in path:
                    if grid[x][y] == material["open"]:
                        grid[x][y] = material["paved"]

                visited_pairs.add((i, j))
                connections_made += 1

        return grid
```

This is a really solid codebase already—you've done the hard part (map generation + candidate path generation). Now we're going to **layer optimization on top**, not replace what you have.

I'm going to give you a **concrete, implementable plan** that plugs directly into your structure and builds the multi-commodity flow model you described.

🔧 Big Picture Integration

Right now your pipeline is:

```
map → POIs → generate paths → draw on grid
```

We're going to change it to:

```
map → POIs → generate candidate edges (A*) → optimization → draw selected e
```

👉 Key shift:
You STOP drawing paths immediately and instead **store them as candidates**.

✅ Step 1 — Extract candidate edges (MODIFY YOUR CODE)

Add this function to `path_generator.py` :

Python Run

```python
def build_candidate_graph(grid, k):
    pois = visitables(grid)
    edges = []  # (i, j, cost, path)

    for i, poi_a in enumerate(pois):
        distances = []
        for j, poi_b in enumerate(pois):
            if i == j:
                continue
            dist = m.dist(poi_a, poi_b)
            distances.append((dist, j, poi_b))

        distances.sort(key=lambda x: x[0])

        count = 0
        for _, j, poi_b in distances:
            if count >= k:
                break

            path = astar(grid, tuple(poi_a), tuple(poi_b))
            if path is None:
                continue
```

✅ Step 2 — Precompute turn angles

We need this for your “sharp corner” penalty.

Python Run

```python
def compute_turn_angle(a, b, c):
    v1 = (a[0]-b[0], a[1]-b[1])
    v2 = (c[0]-b[0], c[1]-b[1])

    dot = v1[0]*v2[0] + v1[1]*v2[1]
    mag1 = m.sqrt(v1[0]**2 + v1[1]**2)
    mag2 = m.sqrt(v2[0]**2 + v2[1]**2)

    if mag1 == 0 or mag2 == 0:
        return 0

    cos_theta = dot / (mag1 * mag2)
    cos_theta = max(-1, min(1, cos_theta))

    angle = m.acos(cos_theta)

    return angle  # radians
```

🚀 Step 3 — Optimization model (NEW FILE: `optimizer.py`)

We'll use PuLP because it's simple and works well for assignments.

Core structure

Python Run

```python
import pulp

def optimize_network(pois, edges, lambda_edges=1.0, lambda_turn=1.0):
    prob = pulp.LpProblem("PathOptimization", pulp.LpMinimize)

    # --- VARIABLES ---
    x = {}  # edge selection
    for e_id, e in enumerate(edges):
        x[e_id] = pulp.LpVariable(f"x_{e_id}", cat="Binary")

    # multi-commodity flow
    f = {}
    n = len(pois)

    for s in range(n):
        for t in range(n):
            if s == t:
                continue
            for e_id, e in enumerate(edges):
                f[(s,t,e_id)] = pulp.LpVariable(f"f_{s}_{t}_{e_id}", lowBound=0,

    # --- OBJECTIVE ---
```

🔗 Flow constraints

Python Run

```python
# flow conservation
for s in range(n):
    for t in range(n):
        if s == t:
            continue

        for v in range(n):
            inflow = []
            outflow = []

            for e_id, e in enumerate(edges):
                if e["j"] == v:
                    inflow.append(f[(s,t,e_id)])
                if e["i"] == v:
                    outflow.append(f[(s,t,e_id)])

            if v == s:
                prob += pulp.lpSum(outflow) - pulp.lpSum(inflow) == 1
            elif v == t:
                prob += pulp.lpSum(inflow) - pulp.lpSum(outflow) == 1
            else:
                prob += pulp.lpSum(inflow) - pulp.lpSum(outflow) == 0
```

🔒 Edge activation constraint

Python Run

```python
for s in range(n):
    for t in range(n):
        if s == t:
            continue
        for e_id in x:
            prob += f[(s,t,e_id)] <= x[e_id]
```

🔥 Step 4 — Turn penalties (IMPORTANT)

We approximate turns at the **edge level**:

Python Run

```
turn_penalty = []

for e1_id, e1 in enumerate(edges):
    for e2_id, e2 in enumerate(edges):
        if e1["j"] == e2["i"]:  # connected

            a = pois[e1["i"]]
            b = pois[e1["j"]]
            c = pois[e2["j"]]

            angle = compute_turn_angle(a, b, c)

            if angle > 0:
                y = pulp.LpVariable(f"turn_{e1_id}_{e2_id}", cat="Binary")

                prob += y <= x[e1_id]
                prob += y <= x[e2_id]

                turn_penalty.append(angle * y)

prob += lambda_turn * pulp.lpSum(turn_penalty)
```

▶️ Step 5 — Solve

Python Run

```
prob.solve(pulp.PULP_CBC_CMD(msg=1))

selected_edges = [e_id for e_id in x if pulp.value(x[e_id]) > 0.5]

return selected_edges
```

🎨 Step 6 — Draw result back onto grid

Python Run

```python
def apply_solution(grid, edges, selected_edges):
    for e_id in selected_edges:
        for (x, y) in edges[e_id]["path"]:
            if grid[x][y] == material["open"]:
                grid[x][y] = material["paved"]
    return grid
```

🧪 How to run everything

In a main file:

Python Run

```python
grid = campus(64, (40.245751,-111.649794), (40.248344,-111.646590))

pois, edges = build_candidate_graph(grid, k=3)

selected = optimize_network(pois, edges,
                            lambda_edges=2.0,
                            lambda_turn=5.0)

grid = apply_solution(grid, edges, selected)

plot_map(grid)
```

⚠️ Performance warning (real talk)

This will:

- Blow up quickly past ~15–25 POIs
- Be slow (minutes is normal)

If needed, we can:

- Prune edges more aggressively
- Reduce commodities
- Cache symmetry (s→t same as t→s)

💡 What makes this impressive

You now have:

- Multi-commodity flow ✅
- Edge selection optimization ✅
- Geometric smoothness penalty ✅
- Real map integration ✅

This is **way beyond standard coursework level**.

👉 Next step

Before we refine:

Send me:

- How many POIs you typically get
- Runtime you're seeing (once you try this)

Then I'll help you:

- Optimize performance
- Improve turn modeling (it can get even cooler)

Quick question for my understanding. In the code you just wrote, where are edges added to the graph? The campus() function doesn't come with any connections already made.

Great question—and this is *exactly* the right thing to be asking, because this is the key conceptual shift.

🧠 Short answer

Edges are not coming from `campus()` at all.

They are created here:

Python Run

```python
pois, edges = build_candidate_graph(grid, k=3)
```

👉 That function is where your “graph” is actually constructed.

🔍 What’s really happening

Your system now has **two layers**:

1. The grid (from `campus()`)

This is:

- A **cost field / environment**
- Not a graph in the optimization sense

It tells you:

- Where you *can* move
- What movement *costs*

But it does **NOT** define connections.

2. The graph (built afterward)

This is created in:

Python Run

```python
build_candidate_graph(grid, k)
```

Inside that function:

Step A — nodes

Python Run

```python
pois = visitables(grid)
```

These are your graph nodes.

Step B — edges (THIS is the answer)

Python Run

```
path = astar(grid, tuple(poi_a), tuple(poi_b))
```

Every time this runs successfully, you create:

Python Run

```
edges.append({
    "i": i,
    "j": j,
    "cost": cost,
    "path": path
})
```

👉 That dictionary entry **IS an edge**.

So each edge contains:

- Start node index (`i`)
- End node index (`j`)
- Cost (length)
- Actual grid path (list of cells)

🧩 Key insight

You are building a graph like this:

```
POI 0 -------- POI 3
   \           /
    \         /
     POI 2 ---
```

But each "line" is:

- Not just a straight line

- A full **A* path through the grid**

⚠️ Important subtlety

Right now, your edges are:

Directed

Because you store:

Python Run

```
"i" → "j"
```

Even though A* paths are reversible.

👉 This matters for flow constraints.

We can:

- Keep them directed (simpler for flow), OR
- Convert to undirected later

🎯 Why this design is powerful

You've separated:

Geometry layer (grid)

- Handles obstacles, buildings, interiors

Graph layer (edges)

- Handles optimization

This is exactly how:

- Road networks
- Game AI navigation
- Robotics planners

are built.

🧪 If you want to verify it

Add this debug print:

Python Run

```python
print(f"# POIs: {len(pois)}")
print(f"# Edges: {len(edges)}")
print(edges[0])
```

You'll see something like:

```
{
  'i': 0,
  'j': 3,
  'cost': 42,
  'path': [(10,5), (11,6), ...]
}
```

🚨 One improvement (you should do this)

Right now you might accidentally duplicate edges:

- (i → j)
- (j → i)

You probably want to avoid that:

Python Run

```python
if i < j:
    edges.append(...)
```

OR explicitly add both directions in a controlled way.

👍 Bottom line

- `campus()` → builds the **world**
- `build_candidate_graph()` → builds the **graph**
- Optimization → selects the **best subset of edges**

If you want, next I can:

- Help you visualize the graph before optimization
- Or cleanly convert this into an undirected + bidirectional flow model (which will make your solver behave better)

> Okay! good news and medium news. The good news is that we don't have too many nodes: even with a 128 by 128 grid with some 20 pois, this thing runs in around 3 seconds. We're doing great there.
>
> Medium news is that we still have lots of lines that are somewhat doubled up on each other: there's lots of extra pavement that doesn't need to be there.

That's actually *excellent* news overall—3 seconds for multi-commodity flow is very workable. The "medium news" is also exactly what we'd expect at this stage.

Right now your solver is doing this:

> "Find cheap ways to connect everything"

But it is **not strongly incentivized to reuse the same paths**—so you get those "parallel highways" instead of one shared road.

Let's fix that.

🧠 Why duplication is happening

Even with:

```
minimize: distance + (# edges)
```

the solver still prefers:

- Two slightly different paths
  over
- One shared path with slightly longer detours

Because:

- There's **no penalty for overlapping geometry**
- Flow commodities don't "see" each other

🔥 The fix: reward path sharing (this is the key upgrade)

We need to encourage:

> "If multiple flows use the same area, that's GOOD."

✅ Option 1 (BEST): Add edge usage cost instead of binary cost

💡 Idea:

Instead of:

```
cost = edge_length * x_e
```

Use:

```
cost = edge_length * (used or not)
```

BUT…

Add a **discount for shared usage**.

🔧 Implementation trick (super effective)

Introduce:

Python Run

```
usage[e] = sum of all flows using edge e
```

Then modify objective:

Python Run

```
cost = edge_length * x_e
       - alpha * usage[e]
```

Why this works:

- More flow on same edge → cheaper overall
- Encourages “highways” instead of duplicates

Code addition

Inside your optimizer:

Python Run

```
usage = {}

for e_id in x:
    usage[e_id] = pulp.lpSum([
        f[(s,t,e_id)]
        for s in range(n)
        for t in range(n)
        if s != t
    ])
```

Then modify objective:

Python Run

```
prob += pulp.lpSum([
    e["cost"] * x[e_id]
    for e_id, e in enumerate(edges)
]) \
+ lambda_edges * pulp.lpSum([x[e_id] for e_id in x]) \
- alpha * pulp.lpSum([usage[e_id] for e_id in x])
```

🔑 Tuning

- `alpha ≈ 0.5–2.0` → mild sharing
- `alpha ≈ 3–10` → aggressive merging

✅ Option 2: Penalize similar edges (geometric redundancy)

Detect edges that:

- Have similar paths
- Are close and parallel

Then penalize:

```
x_e + x_e' ≤ 1   (hard)
```

or:

```
+ beta * x_e * x_e'
```

Simple version (easy win)

Add this before solving:

Python Run

```python
def paths_too_similar(p1, p2, threshold=2):
    set1 = set(p1)
    set2 = set(p2)
    overlap = len(set1 & set2)
    return overlap > threshold
```

Then:

Python Run

```python
for e1_id, e1 in enumerate(edges):
    for e2_id, e2 in enumerate(edges):
        if e1_id >= e2_id:
            continue
        if paths_too_similar(e1["path"], e2["path"]):
            prob += x[e1_id] + x[e2_id] <= 1
```

✅ Option 3 (VERY effective): Increase edge penalty

Right now:

Python Run

```python
lambda_edges = 1.0
```

Try:

Python Run

```python
lambda_edges = 5.0 or even 10.0
```

This alone often:

- Kills redundant paths
- Forces reuse

✅ Option 4: Fix a subtle modeling issue (important)

Right now:

Python Run

```python
f[(s,t,e)] <= x[e]
```

This allows:

- Multiple *different* edges to carry different commodities independently

👉 But nothing forces them to **share**

Upgrade: capacity tightening

Instead of binary:

Python Run

```
f[(s,t,e)] <= x[e]
```

Use:

Python Run

```
sum of all flows on e <= C * x[e]
```

Where `C` is small (like 2–3)

This forces:

- Limited “bandwidth”
- Encourages reuse instead of spreading out

🎯 What I recommend (best combo)

Start with:

1. Add usage reward

2. Increase `lambda_edges`

3. (Optional) add similarity constraints

That combo will give you:

- Fewer paths
- More shared “spines”
- Much less duplication

🧪 What you should expect visually

Before:

```
====    ====
====    ====
```

After:

```
==========
      |
      |
```

👉 One strong shared path with branches

👍 Bottom line

Your model is already correct—it just needs one more ingredient:

> **Make shared paths cheaper than separate ones**

That's the missing piece.

If you want next, I can:

- Help you tune the weights based on actual outputs
- Or upgrade this into a *true Steiner-like network* (which will look insanely clean)

Here's my updated function and how it's being called:

```
def optimize_network(pois, edges, lambda_edges=3.0,
lambda_turn=1.0, alpha=1.0):
    '''Crazy optimizer for doing all the work. like me'''
    prob = pulp.LpProblem("PathOptimization",
pulp.LpMinimize)

    # --- VARIABLES ---
    x = {}  # edge selection
    for e_id, e in enumerate(edges):
```

```
        x[e_id] = pulp.LpVariable(f"x_{e_id}", cat="Binary")

    # multi-commodity flow
    f = {}
    n = len(pois)

    for s in range(n):
        for t in range(n):
            if s == t:
                continue
            for e_id, e in enumerate(edges):
                f[(s,t,e_id)] = pulp.LpVariable(f"f_{s}_{t}_{e_id}",
lowBound=0, upBound=1)



    # check for and incentivize shared edges
    usage = {}

    for e_id in x:
        usage[e_id] = pulp.lpSum([
            f[(s,t,e_id)]
            for s in range(n)
            for t in range(n)
            if s != t
        ])



    # check for and penalize similar edges
    for e1_id, e1 in enumerate(edges):
        for e2_id, e2 in enumerate(edges):
            if e1_id >= e2_id:
                continue
            if paths_too_similar(e1["path"], e2["path"]):
                prob += x[e1_id] + x[e2_id] <= 1

    # --- OBJECTIVE ---
    prob += pulp.lpSum([
        e["cost"] * x[e_id]
        for e_id, e in enumerate(edges)
    ]) \
```

```
    + lambda_edges * pulp.lpSum([x[e_id] for e_id in x]) \
    - alpha * pulp.lpSum([usage[e_id] for e_id in x])

    # flow conservation
    for s in range(n):
        for t in range(n):
            if s == t:
                continue

            for v in range(n):
                inflow = []
                outflow = []

                for e_id, e in enumerate(edges):
                    if e["j"] == v:
                        inflow.append(f[(s,t,e_id)])
                    if e["i"] == v:
                        outflow.append(f[(s,t,e_id)])

                if v == s:
                    prob += pulp.lpSum(outflow) -
pulp.lpSum(inflow) == 1
                elif v == t:
                    prob += pulp.lpSum(inflow) -
pulp.lpSum(outflow) == 1
                else:
                    prob += pulp.lpSum(inflow) -
pulp.lpSum(outflow) == 0

    for s in range(n):
        for t in range(n):
            if s == t:
                continue
            for e_id in x:
                prob += f[(s,t,e_id)] <= x[e_id]

    turn_penalty = []

    for e1_id, e1 in enumerate(edges):
        for e2_id, e2 in enumerate(edges):
```

```
            if e1["j"] == e2["i"]:  # connected

                a = pois[e1["i"]]
                b = pois[e1["j"]]
                c = pois[e2["j"]]

                angle = compute_turn_angle(a, b, c)

                if angle > 0:
                    y = pulp.LpVariable(f"turn_{e1_id}_{e2_id}",
cat="Binary")

                    prob += y <= x[e1_id]
                    prob += y <= x[e2_id]

                    # don't count the angle as a cost if less than
45 degree turn
                    angle_cost = angle
                    if angle < np.pi/4:
                        angle_cost = 0

                    turn_penalty.append(angle_cost * y)

    prob += lambda_turn * pulp.lpSum(turn_penalty)

    prob.solve(pulp.PULP_CBC_CMD(msg=1))

    selected_edges = [e_id for e_id in x if pulp.value(x[e_id])
> 0.5]

    return selected_edges


if __name__ == "__main__":
    grid = campus(128, (40.245751,-111.649794),
(40.248344,-111.646590))
    # grid = simple_space(10, 20, "corners")
    plot_map(grid)

    pois, edges = build_candidate_graph(grid, k=3)
```

```
selected = optimize_network(pois, edges,
                            lambda_edges=2.0,
                            lambda_turn=5.0)

grid = apply_solution(grid, edges, selected)

plot_map(grid)
```

Right now, it's still running fast, but I'm seeing that there are lots of gaps in the connections, meaning, a path may join two POIs, but they as a group aren't connected to any others.

Ahh—this is a really important (and subtle) bug, and your symptoms describe it perfectly:

> "Some POIs are connected in pairs/groups, but the whole system isn't connected."

That means your **flow constraints are not actually enforcing global connectivity** the way we intended.

🧠 Root cause (what's going wrong)

Right now you have:

Python Run

```
f[(s,t,e)] <= x[e]
```

This allows:

- Each commodity (s → t) to independently pick edges
- BUT nothing forces all commodities to use a **consistent shared network**

So what can happen is:

- Commodity (A → B) uses edges {1,2}
- Commodity (C → D) uses edges {7,8}
- Both satisfy flow constraints
- BUT the **union of selected edges is disconnected**

👉 This is the key issue:

> Your model ensures *pairwise connectivity*, not *global network connectivity*

🔥 The fix (CRITICAL): tie flow tighter to edge selection

We need to enforce:

> If an edge is selected, it must actually carry flow
> AND flow must concentrate onto a shared structure

✅ Fix #1: Stronger edge activation constraint

Replace:

Python Run

```
f[(s,t,e_id)] <= x[e_id]
```

with:

Python Run

```
pulp.lpSum([
    f[(s,t,e_id)]
    for s in range(n)
    for t in range(n)
    if s != t
]) <= (n-1) * x[e_id]
```

Why this matters:

- Prevents “ghost edges” (selected but unused)
- Couples all commodities together

✅ Fix #2 (VERY IMPORTANT): enforce at least one global structure

Right now, nothing forces the graph to be a **single connected component**.

Add this constraint:

Python Run

```python
prob += pulp.lpSum([x[e_id] for e_id in x]) >= n - 1
```

This ensures:

- You have enough edges to connect everything

✅ Fix #3 (BEST PRACTICE): switch to single-root flow

Multi-commodity is cool—but it's actually **overkill and less stable** for connectivity.

👉 Instead:

- Pick a root node (say node 0)
- Send flow from root → all others

Replace this:

Python Run

```python
for s in range(n):
    for t in range(n):
```

With:

Python Run

```python
root = 0

for t in range(n):
    if t == root:
        continue
```

Then define:

Python Run

```
f[(t, e_id)]
```

Flow conservation becomes:

Python Run

```python
for t in range(n):
    if t == root:
        continue

    for v in range(n):
        inflow = []
        outflow = []

        for e_id, e in enumerate(edges):
            if e["j"] == v:
                inflow.append(f[(t,e_id)])
            if e["i"] == v:
                outflow.append(f[(t,e_id)])

        if v == root:
            prob += pulp.lpSum(outflow) - pulp.lpSum(inflow) == 1
        elif v == t:
            prob += pulp.lpSum(inflow) - pulp.lpSum(outflow) == 1
        else:
            prob += pulp.lpSum(inflow) - pulp.lpSum(outflow) == 0
```

Why this works better

This guarantees:

- A **single connected tree-like structure**
- All nodes connected to the root
- No disconnected clusters possible

👉 This is the standard trick used in:

- Network design
- MST ILP formulations

⚠️ Big insight

Your current model says:

> "Every pair must be connected somehow"

But NOT:

> "They must all use the same network"

Switching to root flow enforces exactly that.

🎯 What I recommend (clean fix)

Do this:

1. ❌ Remove multi-commodity (s,t)

2. ✅ Use single root flow

3. ✅ Keep your:

- edge penalties
- usage rewards
- turn penalties

🧪 Expected result after fix

Instead of:

```
Cluster A: connected internally
Cluster B: connected internally
(no connection between them)
```

You'll get:

```
One unified network
with shared "spine" paths
```

🚀 Bonus improvement (optional but powerful)

After switching to root flow:

Keep your usage reward, but redefine:

Python Run

```python
usage[e_id] = pulp.lpSum([f[(t,e_id)] for t in range(n) if t != root])
```

This still encourages shared highways.

👍 Bottom line

Your model is **very close**, but:

> Multi-commodity flow ≠ guaranteed single connected network

The fix:

- Use **single-root flow** to enforce connectivity
- Tighten edge-flow coupling

If you want next, I can help you:

- Make the network look even more like real roads (fewer zig-zags)
- Or introduce **Steiner points** so paths merge *midway*, not just at POIs (this would be 🔥 visually)

I'd rather keep the multi-commodity if that's alright. I just ran the new code and it took MUCH longer to run. Are there any obvious syntax issues here that might be making things super long, or have we just finally hit the behavior we were initially expecting? (It is still running, just very slowly)

```
def optimize_network(pois, edges, lambda_edges=3.0,
lambda_turn=1.0, alpha=1.0):
    '''Crazy optimizer for doing all the work. like me'''
    prob = pulp.LpProblem("PathOptimization",
pulp.LpMinimize)

    # --- VARIABLES ---
    x = {}  # edge selection
    for e_id, e in enumerate(edges):
        x[e_id] = pulp.LpVariable(f"x_{e_id}", cat="Binary")

    # multi-commodity flow
    f = {}
    n = len(pois)

    for s in range(n):
        for t in range(n):
            if s == t:
                continue
            for e_id, e in enumerate(edges):
                f[(s,t,e_id)] = pulp.LpVariable(f"f_{s}_{t}_{e_id}",
lowBound=0, upBound=1)


    # check for and incentivize shared edges
    usage = {}

    for e_id in x:
        usage[e_id] = pulp.lpSum([
            f[(s,t,e_id)]
            for s in range(n)
            for t in range(n)
            if s != t
        ])


    # check for and penalize similar edges
    for e1_id, e1 in enumerate(edges):
        for e2_id, e2 in enumerate(edges):
            if e1_id >= e2_id:
```

```
                continue
            if paths_too_similar(e1["path"], e2["path"]):
                prob += x[e1_id] + x[e2_id] <= 1

    # --- OBJECTIVE ---
    prob += pulp.lpSum([
        e["cost"] * x[e_id]
        for e_id, e in enumerate(edges)
    ]) \
    + lambda_edges * pulp.lpSum([x[e_id] for e_id in x]) \
    - alpha * pulp.lpSum([usage[e_id] for e_id in x])

    # flow conservation
    for s in range(n):
        for t in range(n):
            if s == t:
                continue

            for v in range(n):
                inflow = []
                outflow = []

                for e_id, e in enumerate(edges):
                    if e["j"] == v:
                        inflow.append(f[(s,t,e_id)])
                    if e["i"] == v:
                        outflow.append(f[(s,t,e_id)])

                if v == s:
                    prob += pulp.lpSum(outflow) -
pulp.lpSum(inflow) == 1
                elif v == t:
                    prob += pulp.lpSum(inflow) -
pulp.lpSum(outflow) == 1
                else:
                    prob += pulp.lpSum(inflow) -
pulp.lpSum(outflow) == 0

    for s in range(n):
        for t in range(n):
```

```
            if s == t:
                continue
            for e_id in x:
                prob += pulp.lpSum([
                    f[(s,t,e_id)]
                    for s in range(n)
                    for t in range(n)
                    if s != t
                ]) <= (n-1) * x[e_id]

    prob += pulp.lpSum([x[e_id] for e_id in x]) >= n - 1

    turn_penalty = []

    for e1_id, e1 in enumerate(edges):
        for e2_id, e2 in enumerate(edges):
            if e1["j"] == e2["i"]:  # connected

                a = pois[e1["i"]]
                b = pois[e1["j"]]
                c = pois[e2["j"]]

                angle = compute_turn_angle(a, b, c)

                if angle > 0:
                    y = pulp.LpVariable(f"turn_{e1_id}_{e2_id}",
cat="Binary")

                    prob += y <= x[e1_id]
                    prob += y <= x[e2_id]

                    # don't count the angle as a cost if less than
45 degree turn
                    angle_cost = angle
                    if angle < np.pi/4:
                        angle_cost = 0

                    turn_penalty.append(angle_cost * y)

    prob += lambda_turn * pulp.lpSum(turn_penalty)
```

```
    prob.solve(pulp.PULP_CBC_CMD(msg=1))

    selected_edges = [e_id for e_id in x if pulp.value(x[e_id])
> 0.5]

    return selected_edges
```

Yeah—this isn't just "expected slowdown." There's a **specific bug in your constraints that is absolutely nuking performance**. You've accidentally turned one of your constraints into something *massively redundant*.

Let's walk through it.

🚨 The main problem (this is the killer)

This block:

Python Run

```python
for s in range(n):
    for t in range(n):
        if s == t:
            continue
        for e_id in x:
            prob += pulp.lpSum([
                    f[(s,t,e_id)]
                    for s in range(n)
                    for t in range(n)
                    if s != t
                ]) <= (n-1) * x[e_id]
```

❌ What's wrong

Inside the constraint, you **reuse `s` and `t`**, overwriting the outer loop variables.

So this:

Python Run

```
f[(s,t,e_id)]
for s in range(n)
for t in range(n)
```

is actually summing **over ALL commodities**, every single time.

💣 What that means

You are adding:

- ~ $n^2 \times |E|$ constraints
- Each constraint contains ~ n^2 terms

👉 Total size ~ **O($n^4 \cdot |E|$)**

That's catastrophic for the solver.

✅ The fix (simple but huge)

Replace that entire block with:

Python Run

```
for e_id in x:
    prob += pulp.lpSum([
        f[(s,t,e_id)]
        for s in range(n)
        for t in range(n)
        if s != t
    ]) <= (n-1) * x[e_id]
```

✅ Why this works

- Only **one constraint per edge**
- Correctly enforces shared usage
- Reduces complexity by orders of magnitude

⚠️ Secondary performance issues (not bugs, but expensive)

1. Turn penalties = $O(E^2)$

Python Run

```
for e1 in edges:
    for e2 in edges:
```

👉 This explodes fast.

Fix (easy optimization):

Only consider:

- Edges that actually meet
- AND are "likely active"

Add:

Python Run

```
if e1["j"] != e2["i"]:
    continue
```

(you already have this 👍)

But ALSO:

Python Run

```
if e1["i"] == e2["j"]:
    continue  # skip immediate reversals
```

2. Similar path constraints = $O(E^2)$

This is also heavy:

Python Run

```
if paths_too_similar(...)
```

Quick optimization:

Only compare edges with same endpoints:

Python Run

```
if e1["i"] != e2["i"] or e1["j"] != e2["j"]:
    continue
```

This cuts it down massively.

3. Multi-commodity itself

Even *correctly implemented*, you have:

- Variables: $O(n^2 \cdot E)$
- Constraints: $O(n^2 \cdot V)$

With:

- n ≈ 20
- E ≈ ~60

That's already:

- ~24,000 flow variables

👉 So yes, you're now in "real ILP" territory.

🧪 Why it suddenly got slow

Before:

- Your constraints were weaker (and slightly wrong)
- Solver had more freedom → faster

Now:

- Constraints are tighter (good!)
- But bug made them **way too large** (bad)

✅ After fixing, you should see

- Runtime drop from "forever" → back to seconds/minutes

- Still slower than before (expected)
- Much better connectivity behavior

🔥 One more small improvement (worth doing)

Cache this:

Python Run

```
all_pairs = [(s,t) for s in range(n) for t in range(n) if s != t]
```

Then reuse:

Python Run

```
for (s,t) in all_pairs:
```

Cuts Python overhead noticeably.

🎯 Bottom line

❌ The issue is NOT just "expected complexity"

You had a **constraint explosion bug**:

Reusing `s,t` inside the sum created $O(n^4)$ constraints

✅ Fix:

Replace this:

Python Run

```
for s,t:
    for e:
        sum over ALL s,t again
```

with:

Python Run

```
for e:
    sum over ALL s,t once
```

Once you fix that, your model will:

- Run much faster
- Still enforce shared edges
- Keep multi-commodity behavior (as you wanted)

If you want next, I can help you:

- Profile which constraints dominate runtime
- Or reduce model size **without losing the "overkill" feel** (there are some slick tricks for that)